

Scenario 2: Consultation Insights

Production Hackathon — 2 days · teams of 2–4 · presentations at the end of each day

SCENARIO

A DSIT developer built Consultation Insights in a rush the week before the 'Digital Identity in Public Services' consultation closed: a script that runs every response through Claude for a summary, themes, and sentiment, plus a small viewer for the results. The policy team saw the demo and loved it.

Now the plans have escalated. The policy profession wants **every DSIT consultation** run through it. The last consultation received 1,100 responses. The digital identity one is projected to receive **20,000+**, and the department has a statutory duty to publish a summary of responses.

There are two problems. First, **money**: the tool re-sends the full instruction block with every response, one API call per row, on the most capable model available — and it comes out of the CDDO's shared quarterly API budget, which finance is already watching closely after last quarter's overspend. Second, **trust**: the published summary will be read by ministers, journalists, and the respondents themselves. If the tool systematically mislabels sentiment or invents themes, that error compounds 20,000 times into a public document.

The prototype has analysed 40 responses successfully. Nobody has asked it to do more than that. Yet.

THE PROTOTYPE YOU'VE INHERITED

- `analyse.py` — loops through a CSV, one API call per response, full instructions re-sent every time, writes everything to `results.json` in one go at the end
- A crash at row 19,000 of 20,000 loses **everything** — there is no resume, no checkpointing, and re-running re-analyses rows it has already paid for
- If the model returns anything that isn't valid JSON, the script dies (`json.loads` on raw output, no retries, no validation)
- `viewer.py` — a Flask viewer for `results.json`: sentiment counts, theme counts, a table
- No tests, no rate-limit handling, no cost tracking, and two people running it at once will overwrite each other's results
- A 40-row sample dataset is included. It genuinely works — run it first.

YOUR GOAL

Make this production-grade. The question you're really answering: would you sign off running the 20,000-response consultation through this system, with the output feeding a published government document? Spend two days closing the gap between the prototype and a system you would sign off.

By the end of Day 2 you should be able to stand up and say: here is what we found, here is what we prioritised and why, here is the evidence it works, and here is what we would do next.

DIRECTIONS YOU COULD TAKE

These are signposts, not a checklist. You will not do all of them — part of the exercise is deciding what matters most, in what order, and being able to defend that call.

- **Resilience** — checkpointing and resume, retries with backoff, structured output validation, idempotent re-runs. Swapping `results.json` for a **proper database** changes what's possible: resume, concurrent runs, multiple consultations, an audit trail.
- **The cost model** — project the cost of 20,000 responses through the current design. Then attack it: prompt caching on the shared instruction prefix, the Batch API (50% discount for exactly this workload), model choice per task. The before/after table is your headline slide.
- **Cost attribution** — this runs on a shared departmental budget. Tag and record the spend of every run: which consultation, which team, which model. A **standalone spend-monitoring service or database** that other CDDO teams could also use is the ambitious version.
- **Quality you can defend** — hand-label a sample yourselves, measure agreement with the model, and decide what accuracy is acceptable for a published summary. Version your prompts; wire the eval into CI so a prompt 'improvement' that drops agreement fails the build.
- **Throughput** — 20,000 sequential calls at ~2 seconds each is 11 hours. Rate limits, parallelism, and the Batch API all bear on whether this runs overnight or over a week.
- **Productisation** — turn the script into a service: upload a CSV, get a job ID, watch progress, view results per consultation. The viewer becomes something a policy team could use without a developer present.

PRESENTATIONS

- **End of Day 1 (5 minutes):** your priority call — what you found when you audited the prototype, what you decided matters most, and what you've done so far. Alternatively, a short explainer on a concept you researched to unblock yourselves (e.g. how prompt caching pricing actually works, or what the Batch API changes).
- **End of Day 2 (10 minutes):** what you shipped. Demonstrate the difference between the prototype you inherited and the system you're handing back — and name, honestly, what you would do next if you had a third day.

STARTING POINTS

- [Anthropic Batch API docs](#)
- [Anthropic prompt caching docs](#)
- [Anthropic pricing page](#)
- [UK consultation principles](#)

QUESTIONS WORTH ASKING YOURSELVES

1. What would 20,000 responses cost through the current design? What's your best achievable number, and what did each change contribute? Show the maths.
2. If the model mislabels sentiment on 8% of responses, what does that do to the published summary — and how would anyone find out?
3. The consultation summary is itself FOI-able. Could you explain to a requester exactly how a given response was categorised?

4. The run dies at row 19,000. Walk through what happens next in your system, minute by minute.

PRACTICAL NOTES

- **No Docker or Git on your machine?** Fine. Containerisation and version control are directions, not prerequisites — everything runs with plain python. If you want version control without Git, keep dated copies or use a cloud editor; if you want to explore containers, pair with a team member whose machine has Docker.
- **Watch your own API spend.** You have a real API key — this is a FinOps week, so treat your own usage as production usage. Develop against a cheaper model, keep max_tokens sensible, and know roughly what your afternoon of testing cost. Being able to say that number in your presentation is worth more than a feature.